

Netflix Content Analysis

Beginner-to-Advanced Technical, Analytical, and Interview Guide

A complete explanation of the dataset, preprocessing pipeline, dashboard architecture, visualizations, algorithms, code, deployment, limitations, learning path, and interview preparation.

Prepared for

Kshitij

Project state documented: June 18, 2026

TRUTHFUL SCOPE This handbook clearly separates the working browser dashboard from future PRD features. The current system performs descriptive analytics and a simple illustrative extrapolation. It does not train Prophet, ARIMA, Isolation Forest, or any production machine-learning model.

How to Use This Handbook

Start with Parts I-III if you are new. Use Parts IV-VI to understand and defend the implementation. Use Parts VII-IX for learning, interviews, viva, and future engineering.

Part I	Business and product foundations
Part II	Architecture, workflow, files, and technologies
Part III	Dataset, cleaning, preprocessing, engineering, and EDA
Part IV	Visualizations, algorithms, mathematics, statistics, and ML truth
Part V	Every file, every function, and important code walkthroughs
Part VI	Methodology, deployment, challenges, limitations, and roadmap
Part VII	What I must learn to fully understand this project
Part VIII	Interview preparation: 140 answered questions
Part IX	How to explain the project in 30 seconds to 5 minutes

Project Status at a Glance

Implemented	Responsive dashboard, real titles/credits dataset, filters, search, five views, CSV export, CSV/JSON import, country map, contributor joins, descriptive metrics.
Illustrative only	Five-year trend projection and narrative insight cards. These are not trained ML outputs.
PRD future scope	Airflow/dbt medallion pipeline, warehouse, FastAPI, OAuth, Prophet/ARIMA, Isolation Forest, scheduled reports, monitoring, and cloud deployment.

Part I - Business and Product Foundations

What the project is, why it exists, and how it creates value.

1. Executive Summary

Netflix Content Analysis is a browser-based analytical dashboard that turns a supplied catalogue dataset into understandable business views. It helps a user explore what content exists, when it was released, where it was produced, which genres and ratings dominate, and which actors and directors appear most often.

The implemented application is intentionally lightweight: HTML provides structure, CSS provides the visual system, JavaScript performs in-browser data processing, Python prepares the source files, D3 renders geographic geometry, and TopoJSON supplies compressed country boundaries. No database or backend server is required for the current MVP.

The bundled data contains 5,850 titles and 77,801 contributor credits. The preprocessing script joins titles to actors/directors and emits a browser-ready JavaScript catalogue. The dashboard can also import a different CSV or JSON file during the current browser session.

2. Project Objective

- Provide a single place to explore Netflix-like catalogue metadata.
- Reduce manual spreadsheet work by making filters and summaries interactive.
- Show temporal, genre, rating, geographic, runtime, cast, and director patterns.
- Allow a non-programmer to upload another compatible dataset and immediately reuse the dashboard.
- Create a foundation that could later connect to a real API, warehouse, and trained forecasting service.

3. Problem Statement

Raw CSV files are difficult for many stakeholders to use. A person must know formulas, pivot tables, joins, and data-cleaning rules before they can answer simple questions. Data is also easy to interpret inconsistently when each analyst creates a separate spreadsheet.

CORE PROBLEM How can catalogue metadata and contributor credits be converted into a consistent, interactive, and understandable decision-support interface without requiring the user to write code?

4. Real-World Use Cases

Content strategist	Compare genre concentration and release trends before discussing acquisition priorities.
Regional director	Inspect production-country representation and compare regional catalogue mix.
Analyst	Filter, search, and export a clean subset for a report or presentation.
Data scientist	Understand available metadata and identify fields suitable for later modeling.
Student or interviewer	Demonstrate ETL, frontend analytics, geospatial visualization, and honest model-scoping decisions.

Part II - Architecture and Technology

How the system is organized from source data to browser output.

5. Project Architecture

Logical architecture

```
Netflix dataset.zip
|-- titles.csv -----+
|-- credits.csv -----+--> Python preprocessing
                        |-- clean fields
                        |-- join contributors
                        |-- derive region/genre
                        v
                        data/catalogue.js
                        |
index.html + styles.css + app.js + D3 + TopoJSON
                        |
                        v
                        Browser dashboard
                    filters | charts | map | export
                        ^
                        |
                    optional CSV/JSON import
```

This is a static client-side architecture. Static means the browser downloads files and runs the logic locally. Client-side means computation occurs on the user's machine rather than on a remote application server.

Architecture Layers

Source layer	ZIP archive, titles.csv, credits.csv, Natural Earth world boundaries.
Preparation layer	scripts/build_catalogue.py parses, cleans, joins, and serializes records.
Data layer	catalogue.js stores normalized records; countries-110m.js stores map topology.
Presentation layer	index.html defines controls and containers; styles.css controls appearance.
Analytics layer	app.js filters, groups, ranks, calculates metrics, and creates SVG/HTML charts.
Runtime layer	A simple Python HTTP server serves files to the browser.

6. Folder Structure Explanation

index.html	Application shell: sidebar, header, filters, modal, and script loading order.
styles.css	All layout, colors, typography, responsive breakpoints, map, tables, and modal styling.
app.js	Dashboard state, import logic, analytics, chart generation, map rendering, events, and export.
data/titles.csv	5,850 title-level source rows.
data/credits.csv	77,801 actor/director credit rows.
data/catalogue.js	Generated normalized title records consumed directly by the browser.
data/countries-110m.*	Natural Earth country boundaries in TopoJSON and JavaScript form.
scripts/build_catalogue.py	Repeatable ETL/preprocessing script.
vendor/d3.min.js	Local D3 library for projections, selections, CSV parsing, and SVG rendering.
vendor/topojson-client.min.js	Converts compressed TopoJSON geometry to GeoJSON/SVG-ready shapes.
README.md	Quick start, scope, file summary, and production boundaries.
Netflix_Content_Analysis_PRD.docx	Original product requirements and long-term vision.

7. Complete Workflow

1. The source ZIP is copied into the workspace and extracted.
2. Python reads titles.csv and credits.csv with UTF-8-aware CSV parsing.
3. List-like text such as "["drama', 'crime']" is safely parsed into arrays.
4. Credits are grouped by title ID; directors and a limited cast list are attached to each title.
5. Country codes are converted to readable names and broad regions.
6. The normalized records are written into data/catalogue.js.
7. The browser loads libraries, world topology, catalogue data, and app.js in that order.
8. app.js initializes state, populates filters, and renders the Overview view.
9. Each interaction recomputes the active data subset and replaces the view HTML.
10. CSV export serializes the active filtered subset.
11. Optional import reads a user CSV/JSON file, recognizes common aliases, normalizes rows, and replaces the in-memory catalogue.

24-25. Libraries and Technologies Used

Technology	What/why	Advantage	Disadvantage	Alternatives
HTML5	Semantic page structure and native controls.	Simple, accessible, universal.	No analytics behavior by itself.	React, Vue, Svelte.
CSS3	Grid, flexbox, responsive layout, color system, modal styling.	Fast and dependency-free.	Large stylesheets require discipline.	Tailwind, Bootstrap, Sass.
JavaScript ES2020+	State, parsing, filtering, aggregation, rendering, events.	Runs in every modern browser.	Main-thread limits and weaker type safety.	TypeScript, React, Web Workers.
Python 3	Offline ETL and dataset generation.	Excellent CSV/text ecosystem.	Separate preprocessing step.	Node.js, dbt, Spark.
python-docx	Generates this handbook before PDF conversion.	Programmatic Word layout.	Complex layout requires OOXML care.	ReportLab, LaTeX.
D3.js 7.9	CSV parsing, geographic projection, SVG selection/rendering.	Powerful and precise.	Learning curve.	Plotly, Vega-Lite, Leaflet.
TopoJSON Client 3.1	Decompresses shared country boundaries.	Smaller geographic files.	Not beginner-friendly.	GeoJSON, vector tiles.
Natural Earth / world-atlas	Real-world country geometry at 110m resolution.	Public, lightweight, appropriate for world maps.	Not suitable for street detail.	OpenStreetMap, Mapbox.
Python HTTP server	Local static-file serving on port 8000.	Zero configuration.	Not production grade.	Nginx, Vercel, Netlify.

Part III - Data and Analytics

How the two source tables become analytical records.

8. Dataset Analysis

Titles	5,850
Credits	77,801
Movies	3,744
TV shows	2,106
Release-year range	1945-2022
Known directors	4,041 titles
Titles with cast	5,340
Production origins	98 normalized values including Unknown
Primary genres	19

DATASET CAVEAT The dataset does not include Netflix date-added history, true regional availability, viewing hours, subscribers, budgets, or acquisition costs. Therefore the dashboard analyzes release year and production origin, not viewership performance.

9. Explanation of Every Source Column

id	Unique title identifier; join key to credits.csv.
title	Human-readable movie or show name.
type	MOVIE or SHOW; normalized to Movie or TV Show.
description	Synopsis text; preserved for future search or NLP.
release_year	Year of original release; primary temporal field.
age_certification	Audience maturity label such as TV-MA or PG-13.
runtime	Minutes per movie or typical episode, depending on source semantics.
genres	String representation of a list of genre labels.
production_countries	String representation of ISO-like country-code list.
seasons	Number of seasons for shows; blank for movies.
imdb_id	External IMDb identifier.
imdb_score	Average IMDb user score, normally 0-10.
imdb_votes	Count of IMDb votes; useful for confidence/popularity context.
tmdb_popularity	TMDb popularity metric.
tmdb_score	Average TMDb score.

person_id	Unique contributor identifier.
id	Title ID used to join to titles.csv.
name	Actor or director name.
character	Character played; mainly relevant for actors.
role	ACTOR or DIRECTOR; controls which normalized array receives the person.

Normalized Browser Record

genre	First genre, used as the primary grouping dimension.
genres	Full genre array retained for future multi-label analysis.
country	Readable primary production country.
countryCode	Primary source country code.
region	Derived broad business region.
duration	Numeric runtime.
seasons	Numeric season count.
director	Up to three directors joined as a string.
cast	Up to eight actors joined as an array.
imdbScore / imdbVotes	Numeric score and vote count.

10. Data Cleaning Process

Encoding	Files are opened as UTF-8 with BOM handling to avoid broken punctuation and names.
Missing values	Missing genre becomes Unclassified; country/director become Unknown; ratings become Not rated; numeric fields default safely.
List parsing	Python ast.literal_eval converts list-like strings without executing arbitrary code.
Type normalization	MOVIE/SHOW become consistent display labels.
Country normalization	Known codes are mapped to names; unknown codes are preserved.
Numeric conversion	Runtime, seasons, scores, and votes become int/float or null/default values.
Join control	Credits are grouped by title ID; only valid role values are assigned.
Browser import validation	Imported rows require a title and plausible year between 1800 and 2100.

11. Data Preprocessing

Preprocessing means converting raw data into a consistent representation before analysis. The Python script builds a dictionary keyed by title ID, scans credits once, then scans titles once. This avoids repeatedly searching all 77,801 credits for every title.

Core join preparation

```
credits = defaultdict(lambda: {"directors": [], "cast": []})
for row in credits_csv:
    target = credits[row["id"]]
    if row["role"] == "DIRECTOR":
        target["directors"].append(row["name"])
    elif row["role"] == "ACTOR":
        target["cast"].append(row["name"])
```

Time complexity is approximately $O(C + T)$, where C is the number of credit rows and T is the number of title rows. A naive nested loop would be $O(C \times T)$, which would be much slower.

12. Feature Engineering

Primary genre	Uses the first genre for simple mutually exclusive grouping. Advantage: easy charts. Disadvantage: loses multi-genre nuance.
Primary country	Uses the first production country. Advantage: simple map/ranking. Disadvantage: co-productions are underrepresented.
Region	Maps a country to North America, LATAM, APAC, Africa, or Europe/Other.
Contributor counts	Counts appearances per director and actor.
Collaboration pairs	Creates actor $\times$ director keys and counts repeat combinations.
Runtime buckets	Groups movie runtimes into interpretable ranges.
Recent release flag	Counts titles released from 2020 onward.
Illustrative projection	Extrapolates recent annual volume using a deterministic trend formula; not a trained feature/model.

13. Exploratory Data Analysis

- Univariate analysis: counts of type, genre, rating, country, year, and runtime bucket.
- Bivariate analysis: genre by region, actor-director pair frequency, type proportions, and country rankings.
- Temporal analysis: annual title-volume trend over the most recent years.
- Missingness awareness: Unknown and Not rated remain visible rather than silently disappearing.
- Interactive slicing: the same analysis is recalculated after filters and search.

EDA is descriptive. It reveals patterns and data-quality issues but does not prove causality. For example, many US titles do not prove that US content causes subscriber growth.

Part IV - Visualizations, Algorithms, Mathematics, and ML

What calculations are used, what is not used, and why.

14. Visualizations Used

Visualization	Purpose	Advantage	Disadvantage
KPI cards	Immediate totals and proportions.	Fast scanning.	Can oversimplify and currently contains some illustrative comparison text.
Line/area chart	Annual release volume and illustrative outlook.	Shows direction over time.	Area can visually exaggerate magnitude.
Horizontal bars	Genre and market rankings.	Excellent category comparison.	Long labels need space.
Donut chart	Rating/region composition.	Compact share overview.	Harder than bars for precise comparison.
Vertical bars	Runtime bucket distribution.	Good discrete comparison.	Bucket boundaries influence interpretation.
Heatmap	Genre concentration by region.	Finds clusters quickly.	Color perception and scale need care.
World map	Production-origin footprint.	Geographic context.	Marker placement covers selected major markets, not all 98 origins.
Data table	Exact title-level records.	Precise lookup/export context.	Wide tables can require horizontal scrolling.

15-17. Algorithms Used, Reasons, and Alternatives

Algorithm	How it works	Why chosen	Alternative
Filtering	Array.filter tests type, region, genre, year, and search predicates.	Simple and correct for 5,850 rows.	Indexed database queries for millions of rows.
Grouping/counting	Array.reduce builds frequency dictionaries.	One-pass O(n) aggregation.	SQL GROUP BY, d3.rollup, Map.
Ranking	Object.entries + sort descending + slice.	Readable top-N logic.	Heap/partial selection for huge data.
Join	Python dictionary keyed by title ID.	Near-linear join behavior.	pandas merge, SQL JOIN, Spark join.
CSV parsing	Python csv module offline; D3 csvParse for browser imports.	Handles quoted commas and headers.	Papa Parse for streaming/large files.
List parsing	ast.literal_eval offline; conservative JSON/comma parser in browser.	Safer than eval.	Strict JSON source format.
Projection	D3 Natural Earth projection converts longitude/latitude to SVG x/y.	Accurate global display.	Mercator, Robinson, Equal Earth.
TopoJSON conversion	Shared arcs are expanded to GeoJSON features/mesh.	Small file and clean borders.	Raw GeoJSON or vector tiles.
Illustrative forecast	Recent value × deterministic growth plus sinusoidal adjustment.	Demonstrates UI shape without backend.	Prophet, ARIMA, ETS, regression; required for real forecasting.

18-22. Model Training, Tuning, Evaluation, Interpretation, Importance

CRITICAL INTERVIEW ANSWER No machine-learning model is trained in the current implementation. Therefore there is no train/test split, fitted parameter set, hyperparameter search, MAPE result, feature importance output, or production model artifact.

What a Real Forecasting Pipeline Would Do

1. Aggregate genuine monthly Netflix addition dates by genre.
2. Order observations by time and reserve the last 12 months as a holdout set.
3. Fit a baseline seasonal-naive model first.
4. Fit Prophet, ARIMA/SARIMA, exponential smoothing, or gradient-boosted lag models.
5. Tune changepoint prior, seasonality modes, ARIMA p/d/q and seasonal P/D/Q using time-series cross-validation.
6. Evaluate MAE, RMSE, MAPE/SMAPE, interval coverage, and performance against the baseline.
7. Inspect residual autocorrelation and failure periods.
8. Register the selected model, version its data, and monitor drift after deployment.

Hyperparameter Examples

Prophet	changepoint_prior_scale, seasonality_prior_scale, seasonality_mode	Trend flexibility and seasonal strength/form.
SARIMA	p,d,q,P,D,Q,s	Autoregression, differencing, moving average, and seasonal structure.
Isolation Forest	n_estimators, contamination, max_samples	Tree count, expected anomaly rate, sample size.
Gradient boosting	learning_rate, max_depth, n_estimators	Step size, complexity, and number of trees.

Feature Importance

The current descriptive dashboard has no model feature importance. In a future tree model, permutation importance or SHAP could estimate how lagged volume, genre, month, country, rating mix, and seasonality influence predictions. Importance is association with model output, not proof of causation.

29-31. Mathematics, Statistics, and Machine-Learning Concepts

Count	Number of records satisfying a condition: $n = \sum I(\text{condition})$.
Percentage	part / whole $\times 100$; used for movie/show and rating shares.
Mean	$\sum x / n$; suitable for average score when missing values are excluded.
Frequency distribution	Count for each category; foundation of bar and donut charts.
Ranking	Sort categories by count to identify top-N values.
Binning	Convert continuous runtime into intervals; improves readability but loses detail.
Time series	Observations indexed by time; order matters and random train/test split is usually wrong.
Growth rate	$(\text{new} - \text{old}) / \text{old} \times 100$; must not be claimed when comparison data is absent.
MAE	Mean absolute error: average $ \text{actual} - \text{forecast} $.
MAPE	Mean absolute percentage error; unstable when actual values are zero.
Confidence/prediction interval	Range intended to contain an unknown estimate/future observation at a stated rate.
Cosine similarity	Angle-based vector similarity proposed in the PRD for genre mix comparison.
Anomaly detection	Finding observations that differ strongly from normal patterns.
Bias/variance	Underfitting versus sensitivity to training data.
Data leakage	Using future or target information during training, producing unrealistically good results.

Part V - Files, Functions, and Code Walkthrough

A practical guide to what the implementation does.

26. Explanation of Every File

index.html

Application shell: sidebar, header, filters, modal, and script loading order.

styles.css

All layout, colors, typography, responsive breakpoints, map, tables, and modal styling.

app.js

Dashboard state, import logic, analytics, chart generation, map rendering, events, and export.

data/titles.csv

5,850 title-level source rows.

data/credits.csv

77,801 actor/director credit rows.

data/catalogue.js

Generated normalized title records consumed directly by the browser.

data/countries-110m.*

Natural Earth country boundaries in TopoJSON and JavaScript form.

scripts/build_catalogue.py

Repeatable ETL/preprocessing script.

vendor/d3.min.js

Local D3 library for projections, selections, CSV parsing, and SVG rendering.

vendor/topojson-client.min.js

Converts compressed TopoJSON geometry to GeoJSON/SVG-ready shapes.

README.md

Quick start, scope, file summary, and production boundaries.

Netflix_Content_Analysis_PRD.docx

Original product requirements and long-term vision.

27. Explanation of Every JavaScript Function

firstValue()

Finds the first non-empty value matching any accepted column alias. This makes imports tolerant of different header names.

parseListValue()

Converts arrays, JSON-like strings, or comma/pipe/semicolon text into a clean JavaScript array.

normalizeType()

Maps show/series variants to TV Show and all other values to Movie.

normalizeCountry()

Reads the first country and converts common country codes into readable names.

inferRegion()

Returns an explicit region when supplied; otherwise maps known countries to broad regions.

numberValue()

Removes non-numeric characters and safely parses a number with a fallback.

normalizeRecord()

Creates the canonical title object used by every dashboard view and rejects rows without a valid title/year.

Why this matters: it is a central integration point. A defect here can affect multiple views, so it deserves focused tests.

resetDashboardState()

Clears filters/search and rebuilds filter options after a dataset change.

updateDatasetStatus()

Updates the header with the active filename and latest release year.

importDataset()

Reads CSV/JSON, parses rows, normalizes records, reports skipped rows, swaps the active catalogue, and rerenders.

Why this matters: it is a central integration point. A defect here can affect multiple views, so it deserves focused tests.

openImportModal()

Displays the import dialog.

closeImportModal()

Hides the dialog and clears the native file input.

filteredData()

Applies every active filter and the global text search to the catalogue.

Why this matters: it is a central integration point. A defect here can affect multiple views, so it deserves focused tests.

populateFilters()

Builds unique region, genre, and year options from the active dataset.

metricCard()

Returns reusable KPI-card HTML.

panel()

Returns reusable dashboard-panel HTML with title, subtitle, content, width, and optional action.

lineChart()

Calculates SVG coordinates, grid lines, labels, area fill, line path, and tooltip hit areas.

barChart()

Creates horizontal HTML bars or vertical SVG bars depending on the requested orientation.

donutChart()

Calculates cumulative percentages and builds a CSS conic-gradient plus legend.

overviewView()

Calculates type, recent-release, genre, rating, and annual metrics for the overview.

trendView()

Builds release-year history, runtime buckets, recent genre ranking, and illustrative projection.

geographyView()

Calculates country/region counts, market ranking, map panel, donut, and regional heatmap.

worldMap()

Returns the empty accessible container that D3 later fills.

renderWorldMap()

Converts TopoJSON to country paths, fits a Natural Earth projection, and draws borders and markers.

Why this matters: it is a central integration point. A defect here can affect multiple views, so it deserves focused tests.

regionHeatmap()

Counts top genres inside each configured region and converts relative intensity to red opacity.

peopleView()

Counts directors, actors, and actor-director pairings for contributor intelligence.

peopleCards()

Formats contributor leaderboard entries.

libraryView()

Sorts and displays up to 100 matching titles and catalogue summary cards.

libraryTable()

Creates the title/type/genre/origin/year/rating table.

emptyState()

Shows a helpful message when filters return zero records.

render()

Central controller: reads state, calls the active view, inserts HTML, binds tooltips, and triggers map rendering.

Why this matters: it is a central integration point. A defect here can affect multiple views, so it deserves focused tests.

bindTooltips()

Adds mouse listeners to elements with data-tip attributes.

exportCsv()

Escapes filtered values, builds CSV text, creates a Blob URL, and triggers a download.

showToast()

Displays a temporary status notification.

Python Preprocessing Functions

parse_list(value)	Safely parses Python-list-looking text into a list; returns an empty list on malformed input.
region_for(code)	Maps a country code into a broad region.
title_case_genre(value)	Replaces underscores and title-cases a genre label.
Main credits loop	Builds an ID-indexed contributor lookup.
Main titles loop	Normalizes each title and attaches contributor information.
JSON serialization	Writes compact UTF-8 JavaScript data into catalogue.js.

28 & WHAT EACH LINE OF CODE IS DOING

State and catalogue

```
const bundledCatalogue = window.NETFLIX_CATALOGUE || [];  
let catalogue = [...bundledCatalogue];  
let datasetName = "Bundled dataset";
```

Line/step 1: Read the generated catalogue from the global browser object.

Line/step 2: Create a mutable copy so imported data can replace it.

Line/step 3: Store a human-readable name for the active data source.

Filtering

```
return catalogue.filter(item =>  
  (state.type === "All" || item.type === state.type) &&  
  (state.region === "All" || item.region === state.region) &&  
  (state.genre === "All" || item.genre === state.genre) &&  
  (state.year === "All" || item.year === Number(state.year))  
);
```

Line/step 1: filter visits every title.

Line/step 2: Each condition accepts all values when its filter is All.

Line/step 3: Otherwise the record must equal the selected value.

Line/step 4: && means every active condition must be true.

Grouping

```
const group = (items, key) => items.reduce((acc, item) => {  
  const value = typeof key === "function" ? key(item) : item[key];  
  acc[value] = (acc[value] || 0) + 1;  
  return acc;  
}, {});
```

Line/step 1: reduce carries one accumulator object through the array.

Line/step 2: The grouping key can be a property name or a function.

Line/step 3: Missing counts start at zero, then increment.

Line/step 4: The final object maps category to frequency.

Import normalization

```
const normalized = rows.map(normalizeRecord).filter(Boolean);
if (!normalized.length) throw new Error("No valid rows found.");
catalogue = normalized;
resetDashboardState();
render();
```

Line/step 1: map converts each raw row into the canonical schema.

Line/step 2: filter(Boolean) removes rejected null rows.

Line/step 3: The guard prevents replacing the dashboard with empty invalid data.

Line/step 4: The active catalogue is replaced, controls reset, and every view rerenders.

CSV export

```
const blob = new Blob([csv], { type: "text/csv;charset=utf-8" });
const link = document.createElement("a");
link.href = URL.createObjectURL(blob);
link.download = filename;
link.click();
```

Line/step 1: Blob creates an in-memory file.

Line/step 2: A temporary anchor element is created.

Line/step 3: A temporary URL points to the in-memory bytes.

Line/step 4: download supplies the filename.

Line/step 5: click starts the browser download.

Map projection

```
const projection = d3.geoNaturalEarth1()
  .fitExtent([[18, 18], [width - 18, height - 18]], { type: "Sphere" });
const path = d3.geoPath(projection);
```

Line/step 1: Create a Natural Earth projection.

Line/step 2: fitExtent scales and centers the globe inside the SVG margins.

Line/step 3: geoPath converts geographic geometry into SVG path commands.

Python join

```
target = credits[row["id"]]
if row["role"] == "DIRECTOR":
    target["directors"].append(row["name"])
elif row["role"] == "ACTOR":
    target["cast"].append(row["name"])
```

Line/step 1: Use title ID as the dictionary key.

Line/step 2: Route directors into the director list.

Line/step 3: Route actors into the cast list.

Line/step 4: The prebuilt lookup is later attached to title records.

Part VI - Methodology, Deployment, Challenges, and Roadmap

How the work was conducted and what should happen next.

32. Methodology (Research-Paper Style)

Research design

Applied descriptive analytics and software prototyping. The goal was to construct an interactive artefact and evaluate whether supplied metadata could support useful catalogue exploration.

Data source

A supplied archive containing titles.csv and credits.csv. No subscriber, viewership, or confidential Netflix operational data was used.

Population and sample

The working population is the full supplied snapshot: 5,850 titles and 77,801 credits. It is not necessarily the complete historical Netflix catalogue.

Data preparation

Schema inspection, encoding-safe parsing, missing-value handling, type conversion, country/region mapping, contributor grouping, and title-credit join.

Analysis

Frequency analysis, top-N ranking, percentages, runtime binning, release-year trend inspection, regional cross-tabulation, and collaboration counting.

System development

Iterative prototyping from PRD to dependency-free dashboard, followed by real-data integration, layout correction, real-world map replacement, and dataset importer.

Validation

JavaScript syntax checks, record-count checks, HTTP 200 checks for application/data assets, and manual screenshot-driven UI correction.

Ethics and validity

Public metadata only; unknown values are preserved; descriptive associations are not represented as causal effects.

23. Deployment Workflow

Current local	Run <code>python -m http.server 8000</code> in the project folder, then open <code>http://localhost:8000</code> .
Static hosting	Upload HTML/CSS/JS/data/vendor files to GitHub Pages, Netlify, Vercel static hosting, S3+CloudFront, or Nginx.
Production API	Move large data and calculations behind FastAPI; add pagination, caching, authentication, logging, and rate limiting.
Data pipeline	Schedule ingestion/quality/transformation with Airflow and dbt; store curated data in a warehouse.
ML service	Train/version models separately, expose forecasts via API, and monitor accuracy/drift.
CI/CD	Lint/test/build on every commit, deploy preview environments, then promote approved releases.

33. Challenges Faced During Development

Unicode extraction	PowerShell's legacy encoding could not print a PRD symbol. UTF-8 output was explicitly enabled.
Workspace permissions	Files in Downloads required approved copying into the writable workspace.
Dataset mismatch	The supplied dataset lacked date_added and availability; labels and analysis were changed to release year and production origin.
Wide-card layout	A three-column strategy card became unreadable. It was removed and the table width expanded.
Map quality	A rough placeholder map was replaced with Natural Earth/TopoJSON geometry.
Map fetch failure	Browser fetch restrictions caused map loading to fail. Topology was packaged as a local JavaScript global.
Server lifetime	A temporary smoke-test server ended, causing connection refused. A persistent background server was started.
Browser automation	The in-app browser control process was blocked by Windows sandbox permissions, so structural/server tests and user screenshots guided corrections.

34. Limitations of the Current System

- No real Netflix viewership, engagement, subscriber, licensing, or regional-availability data.
- Dataset ends in 2022 and may be incomplete or biased.
- Only the first genre and first country are used for several charts.
- Map markers cover selected major countries rather than every origin.
- The current forecast is illustrative and should not be used for decisions.
- Some KPI comparison phrases are design placeholders rather than calculated historical comparisons.
- All analytics run on the browser main thread; very large imports may become slow.
- Imported data is session-only and is not saved to a database.
- No authentication, authorization, audit log, automated tests, or accessibility certification.
- HTML strings include dataset values; production code should escape untrusted text to prevent injection.

35. Future Improvements

Immediate	Calculate every KPI comparison from data; escape imported text; add importer preview/mapping; add unit tests; support all countries on the map.
Near term	Move to TypeScript/React, add state management, virtualized tables, web workers, and saved filter presets.
Data	Add genuine date-added, availability, viewership, rights, cost, and competitor data with governance.
Backend	FastAPI + warehouse + Redis; pagination; OAuth/SSO; scheduled exports.
ML	Real time-series backtesting, seasonal baselines, Prophet/SARIMA/ETS comparison, anomaly detection, model cards.
Operations	Airflow/dbt, data-quality tests, observability, CI/CD, load testing, WCAG 2.1 AA.

WHAT I MUST LEARN TO FULLY UNDERSTAND THIS PROJECT

A recommended beginner-to-advanced learning path.

Beginner 1	Computer/file basics	Folders, extensions, ZIP/CSV/JSON, terminal, localhost, ports.
Beginner 2	HTML	Elements, attributes, forms, semantic structure, accessibility.
Beginner 3	CSS	Box model, selectors, flexbox, grid, responsive media queries.
Beginner 4	JavaScript fundamentals	Variables, arrays, objects, functions, conditions, loops, DOM events.
Beginner 5	Data basics	Rows, columns, data types, missing values, primary/foreign keys.
Intermediate 1	Modern JavaScript	map/filter/reduce, template strings, async/await, modules, Blob/File APIs.
Intermediate 2	Data analysis	Grouping, percentages, distributions, outliers, binning, EDA.
Intermediate 3	Python	CSV reading, dictionaries, defaultdict, functions, exceptions, JSON.
Intermediate 4	Visualization	Chart choice, scales, SVG, D3 selections, color and accessibility.
Intermediate 5	Geospatial basics	Longitude/latitude, projections, GeoJSON, TopoJSON, choropleths.
Intermediate 6	Software engineering	Git, testing, linting, separation of concerns, security.
Advanced 1	Databases and SQL	JOIN, GROUP BY, indexes, query plans, warehouses.
Advanced 2	Data engineering	ETL/ELT, Airflow, dbt, medallion layers, data quality.
Advanced 3	Statistics	Sampling, uncertainty, confidence intervals, hypothesis tests, causality.
Advanced 4	Time-series ML	Stationarity, lags, seasonality, ARIMA, Prophet, backtesting.

Advanced 5	MLOps	Experiment tracking, model registry, drift, monitoring, reproducibility.
Advanced 6	Production architecture	APIs, OAuth, caching, queues, cloud, CI/CD, observability.

RECOMMENDED ORDER Do not start with machine learning. First become comfortable reading the HTML/CSS/JavaScript and explaining the dataset. Then learn analytics and Python. Add databases and real forecasting only after those foundations.

INTERVIEW PREPARATION

140 questions with answers, organized by difficulty and purpose.

30 Basic Questions

1. What is HTML?

HTML is the language that gives a web page structure. In this project, It defines controls, headings, navigation, and containers in index.html. The main advantage is that it solves a specific, understandable problem. Its limitation is that it does not replace the other layers of the system.

2. What is CSS?

CSS is the language that controls visual presentation. In this project, It creates the grid, responsive design, colors, spacing, modal, and map styling. The main advantage is that it solves a specific, understandable problem. Its limitation is that it does not replace the other layers of the system.

3. What is JavaScript?

JavaScript is the programming language executed by the browser. In this project, It performs filtering, calculations, rendering, import, export, and interaction. The main advantage is that it solves a specific, understandable problem. Its limitation is that it does not replace the other layers of the system.

4. What is CSV?

CSV is a text format where rows are records and commas separate fields. In this project, It is used for titles.csv, credits.csv, imports, and exports. The main advantage is that it solves a specific, understandable problem. Its limitation is that it does not replace the other layers of the system.

5. What is JSON?

JSON is a structured text format based on objects and arrays. In this project, It represents normalized records and imported data. The main advantage is that it solves a specific, understandable problem. Its limitation is that it does not replace the other layers of the system.

6. What is DOM?

DOM is the browser's object representation of HTML. In this project, The app queries elements, changes HTML, and attaches event listeners. The main advantage is that it solves a specific, understandable problem. Its limitation is that it does not replace the other layers of the system.

7. What is Array?

Array is an ordered collection of values. In this project, The catalogue, cast lists, countries, and chart entries are arrays. The main advantage is that it solves a specific, understandable problem. Its limitation is that it does not replace the other layers of the system.

8. What is Object?

Object is a collection of named key/value properties. In this project, Each title and the global state are objects. The main advantage is that it solves a specific, understandable problem. Its limitation is that it does not replace the other layers of the system.

9. What is Function?

Function is a reusable block of logic. In this project, Functions isolate parsing, analytics, charts, views, and events. The main advantage is that it solves a specific, understandable problem. Its limitation is that it does not replace the other layers of the system.

10. What is State?

State is the current values that control the interface. In this project, state stores the active view, filters, and search query. The main advantage is that it solves a specific, understandable problem. Its limitation is that it does not replace the other layers of the system.

11. What is Filter?

Filter is an operation that keeps items satisfying a condition. In this project, `filteredData` creates the active analytical subset. The main advantage is that it solves a specific, understandable problem. Its limitation is that it does not replace the other layers of the system.

12. What is Reduce?

Reduce is an operation that combines many array values into one result. In this project, `group` uses `reduce` to build frequency counts. The main advantage is that it solves a specific, understandable problem. Its limitation is that it does not replace the other layers of the system.

13. What is ETL?

ETL is extract, transform, load. In this project, Python extracts CSVs, transforms rows, and loads `catalogue.js`. The main advantage is that it solves a specific, understandable problem. Its limitation is that it does not replace the other layers of the system.

14. What is Primary key?

Primary key is a unique identifier for a record. In this project, `Title id` identifies a title. The main advantage is that it solves a specific, understandable problem. Its limitation is that it does not replace the other layers of the system.

15. What is Foreign key?

Foreign key is a value referencing another table's primary key. In this project, `credits.csv id` points to `titles.csv id`. The main advantage is that it solves a specific, understandable problem. Its limitation is that it does not replace the other layers of the system.

16. What is Join?

Join is combining related rows across tables. In this project, `Credits` are attached to `titles` using `id`. The main advantage is that it solves a specific, understandable problem. Its limitation is that it does not replace the other layers of the system.

17. What is Missing value?

Missing value is a field with no known value. In this project, The pipeline uses `Unknown`, `Not rated`, `zero`, or `null` depending on meaning. The main advantage is that it solves a specific, understandable problem. Its limitation is that it does not replace the other layers of the system.

18. What is Normalization?

Normalization is making inconsistent values follow one schema. In this project, `MOVIE/SHOW` and `country/genre` formats are standardized. The main advantage is that it solves a specific, understandable problem. Its limitation is that it does not replace the other layers of the system.

19. What is Aggregation?

Aggregation is summarizing many rows. In this project, `Counts by genre, year, rating, and country` are aggregations. The main advantage is that it solves a specific, understandable problem. Its limitation is that it does not replace the other layers of the system.

20. What is Visualization?

Visualization is a graphical encoding of data. In this project, `Charts and maps` make patterns easier to perceive. The main advantage is that it solves a specific, understandable problem. Its limitation is that it does not replace the other layers of the system.

21. What is SVG?

SVG is a vector graphics format represented by DOM elements. In this project, `Line charts and geographic paths` are drawn with SVG. The main advantage is that it solves a specific, understandable problem. Its limitation is that it does not replace the other layers of the system.

22. What is D3?

D3 is a JavaScript library for data-driven documents. In this project, It parses CSV and renders the `Natural Earth map`. The main advantage is that it solves a specific, understandable problem. Its limitation is that it does not replace the other layers of the system.

23. What is GeoJSON?

GeoJSON is a standard geographic feature format. In this project, TopoJSON is converted to GeoJSON-like features for D3. The main advantage is that it solves a specific, understandable problem. Its limitation is that it does not replace the other layers of the system.

24. What is TopoJSON?

TopoJSON is a compressed topology format sharing boundary arcs. In this project, It keeps the world map small. The main advantage is that it solves a specific, understandable problem. Its limitation is that it does not replace the other layers of the system.

25. What is Projection?

Projection is a mathematical conversion from a globe to a flat plane. In this project, Natural Earth converts longitude/latitude to SVG coordinates. The main advantage is that it solves a specific, understandable problem. Its limitation is that it does not replace the other layers of the system.

26. What is Responsive design?

Responsive design is layout that adapts to screen size. In this project, Media queries collapse grids and reveal mobile navigation. The main advantage is that it solves a specific, understandable problem. Its limitation is that it does not replace the other layers of the system.

27. What is API?

API is a contract for software-to-software communication. In this project, A future FastAPI service could deliver filtered metrics. The main advantage is that it solves a specific, understandable problem. Its limitation is that it does not replace the other layers of the system.

28. What is Time series?

Time series is ordered observations indexed by time. In this project, Annual release counts form a simple time series. The main advantage is that it solves a specific, understandable problem. Its limitation is that it does not replace the other layers of the system.

29. What is MAE?

MAE is average absolute prediction error. In this project, It would evaluate a future forecasting model. The main advantage is that it solves a specific, understandable problem. Its limitation is that it does not replace the other layers of the system.

30. What is Data leakage?

Data leakage is using information unavailable at prediction time. In this project, It must be prevented in future temporal modeling. The main advantage is that it solves a specific, understandable problem. Its limitation is that it does not replace the other layers of the system.

30 Intermediate Questions

1. How is HTML used in this project, and what alternative exists?

It defines controls, headings, navigation, and containers in index.html. This choice keeps the MVP readable and lightweight. An alternative would be a more specialized framework or backend implementation when scale, stronger typing, collaboration, or performance becomes more important. The correct choice depends on data volume, team size, deployment constraints, and maintenance needs.

2. How is CSS used in this project, and what alternative exists?

It creates the grid, responsive design, colors, spacing, modal, and map styling. This choice keeps the MVP readable and lightweight. An alternative would be a more specialized framework or backend implementation when scale, stronger typing, collaboration, or performance becomes more important. The correct choice depends on data volume, team size, deployment constraints, and maintenance needs.

3. How is JavaScript used in this project, and what alternative exists?

It performs filtering, calculations, rendering, import, export, and interaction. This choice keeps the MVP readable and lightweight. An alternative would be a more specialized framework or backend implementation when scale, stronger typing, collaboration, or performance becomes more important. The correct choice depends on data volume, team size, deployment constraints, and maintenance needs.

4. How is CSV used in this project, and what alternative exists?

It is used for titles.csv, credits.csv, imports, and exports. This choice keeps the MVP readable and lightweight. An alternative would be a more specialized framework or backend implementation when scale, stronger typing, collaboration, or performance becomes more important. The correct choice depends on data volume, team size, deployment constraints, and maintenance needs.

5. How is JSON used in this project, and what alternative exists?

It represents normalized records and imported data. This choice keeps the MVP readable and lightweight. An alternative would be a more specialized framework or backend implementation when scale, stronger typing, collaboration, or performance becomes more important. The correct choice depends on data volume, team size, deployment constraints, and maintenance needs.

6. How is DOM used in this project, and what alternative exists?

The app queries elements, changes HTML, and attaches event listeners. This choice keeps the MVP readable and lightweight. An alternative would be a more specialized framework or backend implementation when scale, stronger typing, collaboration, or performance becomes more important. The correct choice depends on data volume, team size, deployment constraints, and maintenance needs.

7. How is Array used in this project, and what alternative exists?

The catalogue, cast lists, countries, and chart entries are arrays. This choice keeps the MVP readable and lightweight. An alternative would be a more specialized framework or backend implementation when scale, stronger typing, collaboration, or performance becomes more important. The correct choice depends on data volume, team size, deployment constraints, and maintenance needs.

8. How is Object used in this project, and what alternative exists?

Each title and the global state are objects. This choice keeps the MVP readable and lightweight. An alternative would be a more specialized framework or backend implementation when scale, stronger typing, collaboration, or performance becomes more important. The correct choice depends on data volume, team size, deployment constraints, and maintenance needs.

9. How is Function used in this project, and what alternative exists?

Functions isolate parsing, analytics, charts, views, and events. This choice keeps the MVP readable and lightweight. An alternative would be a more specialized framework or backend implementation when scale, stronger typing, collaboration, or performance becomes more important. The correct choice depends on data volume, team size, deployment constraints, and maintenance needs.

10. How is State used in this project, and what alternative exists?

state stores the active view, filters, and search query. This choice keeps the MVP readable and lightweight. An alternative would be a more specialized framework or backend implementation when scale, stronger typing, collaboration, or performance becomes more important. The correct choice depends on data volume, team size, deployment constraints, and maintenance needs.

11. How is Filter used in this project, and what alternative exists?

filteredData creates the active analytical subset. This choice keeps the MVP readable and lightweight. An alternative would be a more specialized framework or backend implementation when scale, stronger typing, collaboration, or performance becomes more important. The correct choice depends on data volume, team size, deployment constraints, and maintenance needs.

12. How is Reduce used in this project, and what alternative exists?

group uses reduce to build frequency counts. This choice keeps the MVP readable and lightweight. An alternative would be a more specialized framework or backend implementation when scale, stronger typing, collaboration, or performance becomes more important. The correct choice depends on data volume, team size, deployment constraints, and maintenance needs.

13. How is ETL used in this project, and what alternative exists?

Python extracts CSVs, transforms rows, and loads catalogue.js. This choice keeps the MVP readable and lightweight. An alternative would be a more specialized framework or backend implementation when scale, stronger typing, collaboration, or performance becomes more important. The correct choice depends on data volume, team size, deployment constraints, and maintenance needs.

14. How is Primary key used in this project, and what alternative exists?

Title id identifies a title. This choice keeps the MVP readable and lightweight. An alternative would be a more specialized framework or backend implementation when scale, stronger typing, collaboration, or performance becomes more important. The correct choice depends on data volume, team size, deployment constraints, and maintenance needs.

15. How is Foreign key used in this project, and what alternative exists?

credits.csv id points to titles.csv id. This choice keeps the MVP readable and lightweight. An alternative would be a more specialized framework or backend implementation when scale, stronger typing, collaboration, or performance becomes more important. The correct choice depends on data volume, team size, deployment constraints, and maintenance needs.

16. How is Join used in this project, and what alternative exists?

Credits are attached to titles using id. This choice keeps the MVP readable and lightweight. An alternative would be a more specialized framework or backend implementation when scale, stronger typing, collaboration, or performance becomes more important. The correct choice depends on data volume, team size, deployment constraints, and maintenance needs.

17. How is Missing value used in this project, and what alternative exists?

The pipeline uses Unknown, Not rated, zero, or null depending on meaning. This choice keeps the MVP readable and lightweight. An alternative would be a more specialized framework or backend implementation when scale, stronger typing, collaboration, or performance becomes more important. The correct choice depends on data volume, team size, deployment constraints, and maintenance needs.

18. How is Normalization used in this project, and what alternative exists?

MOVIE/SHOW and country/genre formats are standardized. This choice keeps the MVP readable and lightweight. An alternative would be a more specialized framework or backend implementation when scale, stronger typing, collaboration, or performance becomes more important. The correct choice depends on data volume, team size, deployment constraints, and maintenance needs.

19. How is Aggregation used in this project, and what alternative exists?

Counts by genre, year, rating, and country are aggregations. This choice keeps the MVP readable and lightweight. An alternative would be a more specialized framework or backend implementation when scale, stronger typing, collaboration, or performance becomes more important. The correct choice depends on data volume, team size, deployment constraints, and maintenance needs.

20. How is Visualization used in this project, and what alternative exists?

Charts and maps make patterns easier to perceive. This choice keeps the MVP readable and lightweight. An alternative would be a more specialized framework or backend implementation when scale, stronger typing, collaboration, or performance becomes more important. The correct choice depends on data volume, team size, deployment constraints, and maintenance needs.

21. How is SVG used in this project, and what alternative exists?

Line charts and geographic paths are drawn with SVG. This choice keeps the MVP readable and lightweight. An alternative would be a more specialized framework or backend implementation when scale, stronger typing, collaboration, or performance becomes more important. The correct choice depends on data volume, team size, deployment constraints, and maintenance needs.

22. How is D3 used in this project, and what alternative exists?

It parses CSV and renders the Natural Earth map. This choice keeps the MVP readable and lightweight. An alternative would be a more specialized framework or backend implementation when scale, stronger typing, collaboration, or performance becomes more important. The correct choice depends on data volume, team size, deployment constraints, and maintenance needs.

23. How is GeoJSON used in this project, and what alternative exists?

TopoJSON is converted to GeoJSON-like features for D3. This choice keeps the MVP readable and lightweight. An alternative would be a more specialized framework or backend implementation when scale, stronger typing, collaboration, or performance becomes more important. The correct choice depends on data volume, team size, deployment constraints, and maintenance needs.

24. How is TopoJSON used in this project, and what alternative exists?

It keeps the world map small. This choice keeps the MVP readable and lightweight. An alternative would be a more specialized framework or backend implementation when scale, stronger typing, collaboration, or performance becomes more important. The correct choice depends on data volume, team size, deployment constraints, and maintenance needs.

25. How is Projection used in this project, and what alternative exists?

Natural Earth converts longitude/latitude to SVG coordinates. This choice keeps the MVP readable and lightweight. An alternative would be a more specialized framework or backend implementation when scale, stronger typing, collaboration, or performance becomes more important. The correct choice depends on data volume, team size, deployment constraints, and maintenance needs.

26. How is Responsive design used in this project, and what alternative exists?

Media queries collapse grids and reveal mobile navigation. This choice keeps the MVP readable and lightweight. An alternative would be a more specialized framework or backend implementation when scale, stronger typing, collaboration, or performance becomes more important. The correct choice depends on data volume, team size, deployment constraints, and maintenance needs.

27. How is API used in this project, and what alternative exists?

A future FastAPI service could deliver filtered metrics. This choice keeps the MVP readable and lightweight. An alternative would be a more specialized framework or backend implementation when scale, stronger typing, collaboration, or performance becomes more important. The correct choice depends on data volume, team size, deployment constraints, and maintenance needs.

28. How is Time series used in this project, and what alternative exists?

Annual release counts form a simple time series. This choice keeps the MVP readable and lightweight. An alternative would be a more specialized framework or backend implementation when scale, stronger typing, collaboration, or performance becomes more important. The correct choice depends on data volume, team size, deployment constraints, and maintenance needs.

29. How is MAE used in this project, and what alternative exists?

It would evaluate a future forecasting model. This choice keeps the MVP readable and lightweight. An alternative would be a more specialized framework or backend implementation when scale, stronger typing, collaboration, or performance becomes more important. The correct choice depends on data volume, team size, deployment constraints, and maintenance needs.

30. How is Data leakage used in this project, and what alternative exists?

It must be prevented in future temporal modeling. This choice keeps the MVP readable and lightweight. An alternative would be a more specialized framework or backend implementation when scale, stronger typing, collaboration, or performance becomes more important. The correct choice depends on data volume, team size, deployment constraints, and maintenance needs.

30 Advanced Questions

1. Why is browser-side $O(n)$ filtering acceptable?

The dataset has only 5,850 records, so a full scan is quick. At millions of rows, use indexed server queries, pagination, caching, or columnar analytics. The design decision should be justified using correctness, scale, maintainability, security, and user experience.

2. How would you prevent XSS from imported data?

Escape text before inserting it into HTML, avoid raw template interpolation, use `textContent` or a sanitizer, validate MIME/type, and apply a Content Security Policy. The design decision should be justified using correctness, scale, maintainability, security, and user experience.

3. How would you scale the importer?

Stream or chunk parse, use a Web Worker, show progress, enforce file limits, validate schema separately, and upload to a backend for very large files. The design decision should be justified using correctness, scale, maintainability, security, and user experience.

4. Why is a time-ordered split required?

Random splitting lets future patterns leak into training. Train on earlier periods and validate on later periods using rolling-origin evaluation. The design decision should be justified using correctness, scale, maintainability, security, and user experience.

5. How would you evaluate forecast intervals?

Measure empirical coverage: the proportion of actual observations inside the predicted interval, plus interval width and calibration by horizon. The design decision should be justified using correctness, scale, maintainability, security, and user experience.

6. What is the trade-off of primary genre?

It simplifies exclusive counts but discards multi-label information. Multi-hot encoding or fractional counting preserves more nuance. The design decision should be justified using correctness, scale, maintainability, security, and user experience.

7. What is the trade-off of primary country?

It makes one marker/ranking per title but undercounts co-productions. Alternatives are fractional allocation or many-to-many tables. The design decision should be justified using correctness, scale, maintainability, security, and user experience.

8. How would a warehouse schema look?

A title dimension, person dimension, genre dimension, country dimension, and fact/bridge tables for title-person, title-genre, and title-country. The design decision should be justified using correctness, scale, maintainability, security, and user experience.

9. How would Redis help?

Cache expensive aggregate responses by filter key, reducing warehouse/API latency. It adds invalidation and operational complexity. The design decision should be justified using correctness, scale, maintainability, security, and user experience.

10. How would OAuth/SSO work?

The frontend obtains a token from an identity provider; the API validates signatures/scopes before returning protected data. The design decision should be justified using correctness, scale, maintainability, security, and user experience.

11. Why use TopoJSON?

Shared boundaries are encoded once, reducing file size and enabling a border mesh. GeoJSON is simpler but larger. The design decision should be justified using correctness, scale, maintainability, security, and user experience.

12. How do projections distort maps?

Every flat projection distorts area, shape, direction, or distance. Natural Earth balances global aesthetics; equal-area projections suit quantitative choropleths. The design decision should be justified using correctness, scale, maintainability, security, and user experience.

13. How would you test normalizeRecord?

Use unit cases for aliases, missing title, bad year, country arrays, comma-separated genres, numeric strings, and malicious text. The design decision should be justified using correctness, scale, maintainability, security, and user experience.

14. How would you test render?

Test view routing and DOM output with a browser test runner, then visual regression screenshots for layout. The design decision should be justified using correctness, scale, maintainability, security, and user experience.

15. What is a medallion architecture?

Bronze keeps raw data, Silver cleans/conforms it, and Gold stores analysis-ready aggregates. It improves traceability but adds pipeline layers. The design decision should be justified using correctness, scale, maintainability, security, and user experience.

16. Why use dbt?

It manages SQL transformations, tests, lineage, and documentation in version control. It is not an ingestion scheduler. The design decision should be justified using correctness, scale, maintainability, security, and user experience.

17. Why use Airflow?

It schedules and monitors task dependencies. It is powerful but operationally heavier than simple cron or managed orchestration. The design decision should be justified using correctness, scale, maintainability, security, and user experience.

18. How would data quality be measured?

Uniqueness, completeness, validity, referential integrity, freshness, distribution drift, and row-count reconciliation. The design decision should be justified using correctness, scale, maintainability, security, and user experience.

19. What is feature drift?

Input distributions change over time. Monitor metrics such as PSI, KS distance, category shares, and missingness. The design decision should be justified using correctness, scale, maintainability, security, and user experience.

20. What is concept drift?

The relationship between inputs and target changes, causing prediction quality to deteriorate even when input distributions appear stable. The design decision should be justified using correctness, scale, maintainability, security, and user experience.

21. How would SHAP be used?

It decomposes a model prediction into feature contributions. It explains the model, not necessarily the real-world causal process. The design decision should be justified using correctness, scale, maintainability, security, and user experience.

22. Why establish a naive baseline?

A complex model is only useful if it beats a simple rule such as last period or seasonal last year. The design decision should be justified using correctness, scale, maintainability, security, and user experience.

23. When is MAPE inappropriate?

When actual values are zero or near zero, percentage errors explode. Use MAE, RMSE, WAPE, or SMAPE. The design decision should be justified using correctness, scale, maintainability, security, and user experience.

24. How would anomaly detection work?

Create expected seasonal behavior, score residuals or fit Isolation Forest, then validate alerts with domain experts. The design decision should be justified using correctness, scale, maintainability, security, and user experience.

25. What is caching invalidation?

Cached answers become stale after data refresh. Use versioned keys, TTLs, or explicit invalidation after pipeline completion. The design decision should be justified using correctness, scale, maintainability, security, and user experience.

26. What is accessibility risk in charts?

Color-only meaning, poor contrast, missing text alternatives, keyboard-inaccessible interactions, and unreadable responsive layouts. The design decision should be justified using correctness, scale, maintainability, security, and user experience.

27. How would CI/CD improve quality?

Automated linting, tests, builds, security checks, and preview deployments catch defects before release. The design decision should be justified using correctness, scale, maintainability, security, and user experience.

28. How would observability work?

Collect logs, metrics, and traces; define SLIs/SLOs for latency, errors, freshness, and availability; alert on meaningful failures. The design decision should be justified using correctness, scale, maintainability, security, and user experience.

29. What is reproducibility?

A versioned combination of code, data, environment, and parameters should recreate the same output. The design decision should be justified using correctness, scale, maintainability, security, and user experience.

30. How would you migrate to React?

Split cards/charts/views into components, place state in hooks/context, type records with TypeScript, and preserve pure analytics functions. The design decision should be justified using correctness, scale, maintainability, security, and user experience.

30 Project-Specific Questions

1. Why was synthetic data replaced?

The supplied real dataset made conclusions more defensible and exposed real schema constraints.

2. Why are there two CSV files?

Titles contain title metadata; credits form a many-to-one relationship with titles and require a join.

3. Why is catalogue.js generated?

It avoids parsing and joining multi-megabyte CSV files on every page load.

4. Why can the app run without a backend?

All prepared data and logic are static files executed by the browser.

5. Why is localhost required?

Serving over HTTP avoids file-origin restrictions and supports consistent loading of assets.

6. Why did localhost initially fail?

The smoke-test server had stopped, so no process was listening on port 8000.

7. Why was the strategy card removed?

A 3/12-width card forced content into a narrow strip and harmed readability.

8. Why did the first map look unrealistic?

It used hand-drawn continent polygons as a placeholder.

9. How was the map corrected?

Natural Earth TopoJSON, D3 Natural Earth projection, country paths, border mesh, and projected markers.

10. Why did the corrected map still fail once?

Runtime fetching failed in the browser, so topology was loaded as a local JavaScript global.

11. What does `filteredData` return?

Only titles satisfying all active filters and the search predicate.

12. How are top genres calculated?

Group active rows by primary genre, sort counts descending, and keep the leading entries.

13. How is the rating donut calculated?

Count ratings, convert counts to percentages, and create cumulative conic-gradient stops.

14. How are contributors joined?

Python groups credit rows by title ID before attaching directors and cast to each title.

15. Why limit cast and directors?

It controls output file size and dashboard processing, though it sacrifices completeness.

16. How are imported columns recognized?

`firstValue` compares lowercased headers against alias lists.

17. What imported fields are required?

A non-empty title and a plausible release year; type defaults to Movie when unrecognized.

18. Where does imported data live?

Only in JavaScript memory for the current page session.

19. How do you restore the original data?

Open Import data and choose Restore bundled data.

20. What happens to invalid imported rows?

`normalizeRecord` returns null; `filter(Boolean)` removes them and the UI reports the skipped count.

21. How does export work?

It serializes the current filtered subset, creates a text/csv Blob, and triggers a browser download.

22. What is the biggest data limitation?

No viewership or date-added information, so business performance and true addition forecasting cannot be measured.

23. What is misleading in the current UI?

Some growth/velocity comparison texts are illustrative placeholders rather than calculated historical metrics.

24. What is the complexity of filtering?

$O(n)$ for n active catalogue records per render.

25. What is the complexity of top-N ranking?

Counting is $O(n)$; sorting k categories is $O(k \log k)$.

26. How is responsive design implemented?

CSS media queries change grid spans, sidebar behavior, table columns, and button labels.

27. Why use local vendor files?

The app continues working without external CDNs after setup and avoids network variability.

28. What would be your first production refactor?

Escape imported text, calculate all claims, add tests, and separate analytics from HTML generation.

29. What would be your first data-engineering improvement?

Create normalized bridge tables for all genres/countries/contributors rather than selecting only the first.

30. What would make the forecast real?

Genuine monthly date-added data, baseline comparison, rolling validation, trained/versioned models, and monitored errors.

20 Viva Questions

1. State the project in one sentence.

A browser-based catalogue analytics dashboard that preprocesses real title and credit metadata and supports interactive exploration, import, and export.

2. What is your individual contribution?

Translate the PRD into an MVP, integrate the supplied dataset, build preprocessing, analytics views, mapping, import/export, responsive UI, and documentation.

3. What is the dependent variable?

There is no supervised learning target in the implemented system; it is descriptive analytics.

4. What is the unit of analysis?

Usually one title; contributor analysis additionally uses one credit or one actor-director pair.

5. Why is the project not causal?

It observes catalogue metadata without experimental assignment or confounder control.

6. What is the join key?

The id field shared by titles.csv and credits.csv.

7. Why not use Excel?

Excel is useful for ad-hoc analysis, but the dashboard provides repeatable definitions, richer interaction, map rendering, and reusable import/export.

8. Why not use pandas in the browser?

Pandas is a Python library and would require a backend or WebAssembly runtime; JavaScript is native to browsers.

9. What is your strongest engineering decision?

Precompute the title-credit join once, then keep the dashboard static and simple for the current scale.

10. What is your weakest current area?

The forecast and some KPI comparisons are illustrative, not data-derived production analytics.

11. How do you ensure correctness?

Schema checks, row-count reconciliation, syntax checks, HTTP asset checks, explicit missing-value rules, and future automated tests.

12. What happens with a malformed file?

The importer catches parse errors, rejects invalid rows, and displays an error without replacing the active catalogue.

13. What is one security risk?

Unescaped imported text inserted into HTML could permit XSS; production code should use `textContent`/sanitization.

14. What is one performance risk?

Large imports cause repeated main-thread $O(n)$ filtering and DOM rendering.

15. What is one ethical consideration?

Avoid claiming viewer preferences, demand, or performance from metadata alone.

16. How would you make results reproducible?

Version the source archive, preprocessing code, generated catalogue, dependencies, and deployment commit.

17. How would you validate region mapping?

Use ISO country libraries, unit tests, and reconciliation of unmapped/Unknown codes.

18. Why Natural Earth?

It provides lightweight, recognized global boundary data appropriate for a dashboard overview.

19. What would you demonstrate live?

Import a small CSV, filter it, inspect the geographic and library views, export the subset, then restore bundled data.

20. What is the final conclusion?

The MVP successfully makes catalogue metadata explorable, but reliable strategic prediction requires richer data and a real validated backend/ML pipeline.

EXPLAIN THIS PROJECT TO AN INTERVIEWER

Ready-to-use versions for different time limits.

30-second version

I built a Netflix catalogue analytics dashboard using HTML, CSS, JavaScript, Python, D3, and TopoJSON. I processed 5,850 titles and 77,801 cast/director credits, joined and normalized them into a browser-ready dataset, and created interactive overview, trend, geographic, contributor, and library views. The app also supports filtering, search, CSV export, and importing another CSV or JSON dataset. It is a descriptive MVP; the forecasting screen is illustrative, not a trained production model.

1-minute version

This project converts raw Netflix-like metadata into an interactive decision-support dashboard. The source has separate title and credit tables, so I wrote a Python preprocessing pipeline to parse list fields, normalize types and countries, group actors/directors by title ID, and emit a compact browser dataset. The frontend is dependency-light: semantic HTML, responsive CSS, and JavaScript for state, filtering, aggregation, SVG charts, import, and export. D3 and TopoJSON render a real Natural Earth world map. I made deliberate corrections when the real dataset lacked date-added and availability fields, so the final analysis uses release year and production origin. For production, I would add a warehouse/API, tests, security hardening, and real time-series backtesting.

3-minute version

The original PRD described a large 20-week platform with pipelines, a warehouse, APIs, machine learning, and reporting. I scoped that into a working frontend MVP while preserving the main analytical user journey. The supplied archive contains 5,850 titles and 77,801 contributor credits. I first inspected the schema and discovered it differs from the PRD dataset: it has release year, runtime, seasons, IMDb/TMDb metrics, genres, production countries, actors, and directors, but no Netflix date-added or true regional availability. I therefore changed the dashboard language and avoided pretending those fields existed. A Python ETL script reads both CSVs, safely parses list-like values, creates a dictionary keyed by title ID, joins directors and cast in near-linear time, derives country/region and primary genre, and generates catalogue.js. In the browser, app.js stores filter state and recomputes descriptive analytics using filter, reduce, grouping, sorting, and top-N selection. Reusable rendering functions create KPI cards, line and bar charts, donut charts, heatmaps, tables, and a Natural Earth map rendered with D3 and TopoJSON. I also added CSV/JSON import with common column aliases, row validation, skipped-row reporting, and restore-original-data behavior. The major limitation is that the forecast is illustrative. A defensible ML version would need genuine monthly addition or demand data, time-based validation, naive baselines, Prophet/SARIMA comparison, and monitored errors.

5-minute version

I started from a product requirements document for a Netflix Content Analysis Platform. The long-term vision included a medallion data pipeline, a warehouse, FastAPI, forecasting, anomaly detection, reporting, and an API. Rather than claim all of that was built, I delivered a self-contained analytical MVP and documented the production path. The data engineering problem was important because titles and contributors are separate. I used Python's csv module and ast.literal_eval, grouped credits in a defaultdict keyed by title ID, limited contributor arrays for browser size, normalized categorical and numeric fields, derived primary country/region and genre, and serialized 5,850 normalized records. The frontend separates structure, presentation, and behavior into index.html, styles.css, and app.js. A central state object controls view, type, region, genre, year, and search. filteredData performs an O(n) scan, which is acceptable at this scale. group uses reduce for one-pass frequency distributions, topEntries ranks categories, and individual view functions calculate the values needed for each screen. Charts are implemented with SVG or CSS, while the map uses a Natural Earth projection and TopoJSON boundaries. During development I corrected several real issues: Unicode extraction, workspace permissions, the temporary server stopping, a narrow card breaking layout, an unrealistic placeholder map, and browser fetch restrictions. The importer now accepts CSV or JSON, recognizes common aliases, validates title and year, normalizes each row into the same canonical schema, and rerenders every view. For production I would address XSS escaping, automated tests, all-country markers, calculated KPI comparisons, large-file workers, a normalized database schema, APIs, authentication, caching, observability, and real forecasting with rolling backtests. The strongest lesson is scope honesty: this is a good descriptive analytics product, but metadata alone cannot prove audience demand or content performance.

Final Defense Checklist

- Say exactly which features are implemented.
- Do not describe the illustrative projection as trained AI.
- Know the two source tables and their join key.
- Be able to explain filter, reduce, dictionary join, and Natural Earth projection.
- State at least three limitations without being prompted.
- Describe the first production improvements: security, testing, calculated KPIs, backend, and real ML validation.
- Demonstrate import, filtering, map, export, and restore flow.

Glossary

MVP	Minimum viable product: smallest useful version that tests the core experience.
PRD	Product requirements document.
EDA	Exploratory data analysis.
ETL	Extract, transform, load.
DOM	Document Object Model.
SVG	Scalable Vector Graphics.
API	Application Programming Interface.
XSS	Cross-site scripting.
SSO	Single sign-on.
CI/CD	Continuous integration and continuous delivery/deployment.
SLA/SLO/SLI	Service commitment, objective, and measured indicator.
MAPE/MAE/RMSE	Common forecast error metrics.

END OF HANDBOOK

You should now be able to understand the project, demonstrate it, explain its engineering decisions, defend its limitations, and describe a credible path from MVP to production.

Prepared for Kshitij